

Lab 12: Noise a Point, Then Take One Reverse Step

A 2-D schedule, a known score, one denoising update.

Work in a **Jupyter** notebook with **NumPy** (PyTorch is optional). Do notes **12.1–12.2** first. No U-Net and no pretrained weights.

First time you implement this. The lecture notes this week already answered what the method is, why it exists, the architecture, the formula, and the tradeoffs. Read that first-time block before these exercises. This lab is the first time you **compute** it, not the first definition.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
x0 = np.array([1.5, -0.8])      # one 2-D data point
T = 40
beta = np.linspace(1e-4, 0.20, T + 1)
beta[0] = 0.0
alpha = 1.0 - beta
alphabar = np.cumprod(alpha)

def q_sample(x0, t, eps):
    a = alphabar[t]
    return np.sqrt(a) * x0 + np.sqrt(1.0 - a) * eps
```

The schedule is constructed. You will add noise, then take **one** reverse step with the true posterior mean (you know x_0).

1. Forward path

1. Print `alphabar[1]`, `alphabar[T//2]`, `alphabar[T]`. They should fall from near 1 toward 0.
2. Sample a chain: start at `x0`, for `t = 1 ... T` draw `eps ~ N(0, I)` and set `xt = q_sample(x0, t, eps)`. Store every `xt`. (Independent $x_t \mid x_0$ draws are enough; you do not need the Markov simulation.)
3. Scatter the path in 2-D, color by t , and mark x_0 . At large t the points should look like isotropic noise.

2. True posterior mean for one step

The Gaussian $q(x_{t-1} \mid x_t, x_0)$ has mean

$$\tilde{\mu} = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t.$$

```
def posterior_mean(x0, xt, t):
    c0 = np.sqrt(alphabar[t - 1]) * beta[t] / (1.0 - alphabar[t])
    c1 = np.sqrt(alpha[t]) * (1.0 - alphabar[t - 1]) / (1.0 - alphabar[t])
    return c0 * x0 + c1 * xt
```

Pick a mid t (try `t = 20`). Draw one `eps`, form `xt`, then `xhat = posterior_mean(x0, xt, t)`.

1. Print $\|x_t - x_0\|$ and $\|\hat{x} - x_0\|$. The second should be smaller.
 2. Repeat the draw 50 times at the same t . Report mean distances before and after the step.
-

3. Score form (same geometry)

For this spherical forward kernel the noise is $\varepsilon = (x_t - \sqrt{\bar{\alpha}_t} x_0) / \sqrt{1 - \bar{\alpha}_t}$. Plug that **known** ε into the DDPM mean from note **12.2** (no extra z) and check it matches `posterior_mean` up to small numeric error at your t .

Write four sentences: what $\bar{\alpha}_t$ does, why this lab's reverse step is an oracle, what a trained ε_θ would replace, and why a GAN does not need a T -step chain to sample.